Soft Value-Based Decoding for Guided Molecular Generation on a Frozen Graph-Diffusion Model

Abel Le · CIS 6930 Generative AI, Fall 2026 · **Enrollment: graduate (6930)** · Path A (own topic)

1 Question

Reward-guided generation usually fine-tunes the generator (e.g., RL fine-tuning of diffusion models [2]), which is costly and needs differentiable or on-policy training. Soft Value-Based Decoding (SVDD) [5] instead steers a frozen diffusion model at inference time: at every reverse step it draws M candidates, scores each with a learned value function, and keeps one. Whether this works for *discrete graph* diffusion on molecules, and whether its extra per-step compute beats the simplest alternative of spending the same compute on independent samples and keeping the best, is untested. **This project asks whether SVDD with a learned value function, applied to a frozen pretrained DiGress model, raises the mean synthetic-accessibility and QED reward of generated QM9 molecules more than best-of-N sampling at an equal number of denoiser evaluations, without lowering validity, and how that advantage scales with the number of candidates M.**

2 Primary Paper and Background

Primary: SVDD [5] shows that a soft value function $v_t(x_t) = \alpha \log \mathbb{E}[e^{r(x_0)/\alpha} \mid x_t]$ lets a pretrained diffusion model sample from a reward-tilted distribution with no fine-tuning, on images, DNA/RNA and molecules. **Supporting:** DiGress [7] is a discrete graph diffusion model whose denoiser predicts clean atom and bond types, with released QM9 weights. DDPO [2] is the fine-tuning route to reward optimization that SVDD avoids. Gao et al. [4] show that best-of-n selection is a strong, compute-matched reward-optimization baseline, which motivates my comparison. **Gap:** the SVDD authors' molecule code (`hyanan16/SVDD-molecule`) documents a separate pipeline (docking-style rewards), not DiGress or QM9, and I know of no SVDD-vs-best-of-N comparison at matched denoiser calls on a discrete graph model. I measure exactly that. This is new work started this semester, not prior research.

3 Method

Pipeline (all inference-time; DiGress weights frozen). (1) *Proposal:* a thin wrapper around the DiGress QM9 checkpoint exposes $p_\theta(G_{t-1} \mid G_t) = \sum_{G_0} q(G_{t-1} \mid G_t, G_0)\, p_\theta(G_0 \mid G_t)$ and returns M i.i.d. candidates per molecule per step (T=500). DiGress was trained with cross-entropy on node/edge types and is not retrained. (2) *Value function* $\hat{v}_\phi(G_t, t)$, trained by MSE regression onto $r(G_0)$ using 10k unguided rollouts (the SVDD-MC estimator); two architectures: an MLP on pooled atom/bond-type histograms plus a timestep embedding, and a 3-layer message-passing GNN on the (soft) graph. (3) *Selection:* keep $\arg\max_m \hat{v}_\phi(G_{t-1}^{(m)})$ (or sample $\propto e^{\hat{v}/\alpha}$). A training-free variant, SVDD-PM, scores a candidate by $r(\hat{G}_0)$ of its decoded denoiser prediction. (4) *Rewards:* SA $= (10 - \text{SA})/9$ [3] and QED [1] via RDKit, invalid $\to 0$. Value functions are trained per reward. **Stretch:** a docking-proxy reward, dropped if not started by Nov 6 (see Risks).

4 Data

QM9 [6]: 133,885 molecules (up to 9 heavy atoms; MoleculeNet CSV copy), public, released under CC0 on figshare, no personal data. The pretrained DiGress model already encodes QM9, so I *generate* molecules rather than train on them; QM9 is used as the novelty reference and for three seeded (42), disjoint 1k-molecule index splits (train/val/test, committed) used to sanity-check reward code and value-function predictions at t=0. Value-function training data are unguided DiGress trajectories, so no test molecule leaks into training. Each evaluation run generates N=2,000 molecules.

5 Evaluation Plan

Metrics: mean and top-10% reward, validity (RDKit-sanitizable, connected), uniqueness, novelty vs. QM9, diversity (mean pairwise ECFP4 Tanimoto distance), denoiser calls and wall-clock. **Seeds:** 0, 1, 2; report mean±std. **Baselines:** (B1) unguided DiGress (M=1); (B2) *best-of-N at matched compute*: generate M unguided molecules per slot and keep the highest-reward one, so cost equals SVDD with the same M; (B3) SVDD-PM without a learned value. **Controlled comparisons:** (A1) $M \in \{1, 5, 10, 20\}$ for SA and QED; (A2) value estimator $\in$ {PM, MLP, GNN} at M=10. I will not run DDPO-style fine-tuning (needs training, outside the budget) or DiGress's own regressor guidance (it needs a retrained property regressor and is a different question). **Falsifiable outcome:** the hypothesis is wrong if, at M=10, SVDD-MLP's mean-reward gain over B2 is not larger than twice the across-seed standard deviation, *or* if SVDD reaches its gain only with validity more than 5 points below B1. A null or negative result will be reported and analyzed.

6 Compute Plan

Where: GAIVI (Slurm batch, scripts in repo), Colab/local GPU as backup; I already have a GAIVI account and will run a first trivial job in week 1. **Estimate** = (steps × s/step)/3600 × runs, with T=500 steps, batches of 512 graphs, an assumed 0.1 s per batch-step (QM9 graphs have ≤9 nodes; *to be measured in week 1*) and a 1.5× overhead for featurizing and scoring candidates. Main sweep: 2000×(1+5+10+20) = 72k graphs → 141 batches; 141×500×0.1×1.5/3600 = 2.9 h per (reward, seed); × 2 rewards × 3 seeds = 17.4 h. Value-fn data (10k rollouts, 20×500×0.1/3600 = 0.28 h) × 3 seeds = 0.8 h; value-fn training 0.5 h × 12 (2 rewards, 2 archs, 3 seeds) = 6 h. A2 ablation: 2000×10 graphs → 0.83 h per run; 2 new variants (PM, GNN) × 2 rewards × 3 seeds = 10 h. B2: 2000 × 35 = 70k graphs → 137×500×0.1/3600 = 1.9 h per seed, reward-independent, ×3 = 5.7 h. Total ≈ 40 h; with a 25% rerun margin, **≈50 GPU-hours**. **Single-GPU fallback (<2 h):** N=1000, $M \in \{1, 10\}$, one seed, MLP value, SA reward: 11k graphs ≈ 0.5 h plus 1k rollouts and training ≈ 0.2 h. Checkpoints are written every 50 batches; every run logs config, seed, git commit and metrics.

7 Milestones (approval Oct 12 → presentation, targeting Slot C, Fri Dec 4)

Oct 12–18	Run first trivial GAIVI job; pin DiGress/SVDD commits; load QM9 checkpoint and reproduce DiGress validity; measure s/batch-step.
Oct 19–25	Finish `propose()` wrapper; B1 unguided baseline over 3 seeds; SVDD-PM end to end.
Oct 26–Nov 1	Collect rollouts; train MLP value fn; complete the <2 h fallback run.
Nov 2–8	GNN value fn; **Nov 6 check-in and go/no-go** on GNN and docking proxy.
Nov 9–15	Main M sweep (A1), SA and QED, 3 seeds.
Nov 16–22	B2 best-of-N and value-estimator ablation (A2); first full results table.
Nov 23–29	Failure analysis (where the value fn mis-ranks), figures, seed variance checks.
Nov 30–Dec 4	Slides, rehearse code walk-through; **present Dec 4**. Report and code due Dec 11.

8 Risk and Fallback

(i) If the DiGress QM9 checkpoint (hosted on a personal drive) cannot be loaded or reproduce validity by Oct 25, I will train a reduced DiGress on QM9 with the repository's training script (budget ≤10 extra GPU-h). (ii) If the GNN value function does not beat the MLP on validation MSE by Nov 6, I drop it and keep PM vs. MLP as ablation A2. (iii) If measured s/batch-step is more than 3× my estimate, I cut N to 1000 and the M grid to $\{1, 10, 20\}$ so the total stays near 50 GPU-h. (iv) If SVDD does not beat B2, the deliverable becomes a negative-result analysis of the value function's ranking error.

9 Contribution Table (full version in `CONTRIBUTIONS.md`)

Component	Origin	What I changed
DiGress model	`cvignac/DiGress` @ `bdfa10c` (MIT), QM9 checkpoint	Weights and architecture unmodified; added a candidate-proposal wrapper (`vgmg/models/digress.py`)
SVDD algorithm	Li et al. [5]; `hyanan16/SVDD-molecule` @ `455e4d4` read for reference only	Own implementation from the paper (`vgmg/rl/svdd.py`); ported to discrete graph diffusion
Value functions	Mine	MLP, GNN, and PM estimators; training loop
Rewards, metrics	RDKit QED; SA score from RDKit Contrib	Own wrappers and rescaling; validity/novelty/diversity code is mine
Best-of-N, ablation harness	Mine	Compute-matched baseline; sweep scripts and logging

10 AI Tool Declaration

I used Claude Code to scaffold the repository, draft unit tests, and edit this proposal. The research question, experimental design, baselines, and ablation choices are mine; I will write and understand the DiGress wrapper and SVDD code, and all reported numbers will come from my own runs.

References

[1] G. Richard Bickerton, Gaia V. Paolini, Jérémy Besnard, Sorel Muresan, and Andrew L. Hopkins. Quantifying the chemical beauty of drugs. *Nature Chemistry*, 4:90–98, 2012.

[2] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Peter Ertl and Ansgar Schuffenhauer. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of Cheminformatics*, 1(8), 2009.

[4] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, pages 10835–10866, 2023.

[5] Xiner Li, Yulai Zhao, Chenyu Wang, Gabriele Scalia, Gokcen Eraslan, Surag Nair, Tommaso Biancalani, Shuiwang Ji, Aviv Regev, Sergey Levine, and Masatoshi Uehara. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. In

Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2408.08252.

[6] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.

[7] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. DiGress: Discrete denoising diffusion for graph generation. In *International Conference on Learning Representations (ICLR)*, 2023.